

# Informe Técnico: Arquitectura Híbrida de Recuperación Aumentada por Generación (RAG) con Grafo de Conocimiento

Team Kepler

10 de agosto de 2026

## Índice

|                                                                                        |          |
|----------------------------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                                 | <b>2</b> |
| <b>2. Objetivos</b>                                                                    | <b>2</b> |
| 2.1. Objetivo General . . . . .                                                        | 2        |
| 2.2. Objetivos Específicos . . . . .                                                   | 2        |
| <b>3. Metodología y Flujo de Trabajo</b>                                               | <b>2</b> |
| 3.1. Roles y Distribución del Trabajo . . . . .                                        | 2        |
| 3.1.1. Módulo 1: Extracción y Procesamiento de Texto (Nombre del Integrante) . . . . . | 3        |
| 3.1.2. Módulo 2: Indexación Vectorial (Nombre del Integrante) . . . . .                | 3        |
| 3.1.3. Módulo 3: Grafo de Conocimiento y Fusión RRF (Tu Nombre) . . . . .              | 3        |
| 3.1.4. Orquestación y Despliegue (Líder del Equipo) . . . . .                          | 3        |

# 1. Introducción

En el marco del presente reto tecnológico, este informe detalla la conceptualización, diseño y despliegue de un pipeline de Recuperación Aumentada por Generación (RAG). El objetivo central fue superar las limitaciones de la búsqueda vectorial tradicional mediante la integración de un Grafo de Conocimiento semántico. Esta arquitectura híbrida permite no solo recuperar información basada en la similitud de contextos multilingües, sino también priorizar documentos mediante relaciones topológicas exactas entre entidades clave detectadas en las consultas de los usuarios.

## 2. Objetivos

### 2.1. Objetivo General

Desarrollar e implementar un sistema RAG híbrido, modular y escalable que fusione técnicas de recuperación vectorial (FAISS) con la precisión de un Grafo de Conocimiento estructurado, optimizando la extracción de información en un corpus de documentos multilingüe.

### 2.2. Objetivos Específicos

- Extraer y procesar texto e imágenes de un corpus documental aplicando técnicas de OCR y segmentación (chunking) controlada (máximo 250 palabras por fragmento).
- Generar embeddings multilingües de alta calidad e indexarlos utilizando la librería FAISS para búsquedas por similitud semántica.
- Construir un Grafo de Conocimiento extrayendo entidades nombradas (NER) mediante modelos de procesamiento de lenguaje natural (Hugging Face) e identificando sus relaciones de co-ocurrencia.
- Implementar el algoritmo de fusión de rangos recíprocos (RRF) para combinar los scores vectoriales y topológicos, retornando los resultados en un formato JSONL estricto.

## 3. Metodología y Flujo de Trabajo

Para garantizar la agilidad y modularidad del proyecto, el equipo adoptó un flujo de trabajo colaborativo centralizado en GitHub. El desarrollo se estructuró en módulos independientes ejecutados secuencialmente a través de un orquestador principal (`main.py`). Esto permitió el trabajo en paralelo y la validación aislada de cada componente antes de su integración final.

### 3.1. Roles y Distribución del Trabajo

A continuación, se detallan las responsabilidades asumidas por cada integrante del equipo, reflejando la arquitectura técnica del pipeline:

### 3.1.1. Módulo 1: Extracción y Procesamiento de Texto (Nombre del Integrante)

**Archivos responsables:** `extractor.py`, `procesador_texto.py`

Encargado de la lectura del corpus en crudo, la aplicación de OCR (PyTesseract) para la extracción de texto en imágenes, y la normalización del contenido. Su función principal fue garantizar la segmentación del texto en fragmentos (chunks) respetando el límite de 250 palabras y estructurando la metadata inicial.

### 3.1.2. Módulo 2: Indexación Vectorial (Nombre del Integrante)

**Archivo responsable:** `indexador.py`

Responsable de transformar el texto procesado en representaciones vectoriales utilizando modelos de *Sentence Transformers*. Este rol implementó la base de datos FAISS (`index.faiss`) para almacenar los embeddings y generó el almacén persistente de metadatos (`metadata.jsonl`).

### 3.1.3. Módulo 3: Grafo de Conocimiento y Fusión RRF (Tu Nombre)

**Archivos responsables:** `constructor_grafo.py`, `generador.py`

Encargado del componente híbrido del sistema. Implementó un pipeline NER multilingüe para construir el archivo `grafo.graphml` mapeando entidades (Organizaciones, Locaciones, Personas). Asimismo, desarrolló el algoritmo de recuperación final, cruzando las predicciones de FAISS con los vecinos del grafo mediante Fusión RRF y aplicando *Max Pooling* a nivel de documento.

### 3.1.4. Orquestación y Despliegue (Líder del Equipo)

**Archivo responsable:** `main.py`

Encargado de unificar los scripts, gestionar las dependencias del entorno virtual, asegurar que los datos fluyan correctamente de un módulo a otro a través de subprocesos, y realizar la entrega oficial del repositorio.